MiniOS Individual Report: Command-Line Interface

Name: Zenadine Zidane C. Pablo
OS Name: karlOS
Role: CLI Developer and error handling
Date: 10-20-2026

1. Introduction

Our group built MiniOS, a simple command-line operating system simulator in Python that runs in Windows Command Prompt. It is only a simulation, so it uses Python lists, dictionaries, and classes instead of real hardware. I was the CLI developer, so I built the entry point (main.py) and the shell (shell.py), which is where the user types commands.

2. What I Built

- main.py: shows the boot screen, creates the shared object, and starts the shell.
- shell.py: reads, parses, and runs commands.
- kernel.py: a simple container for shared data (current user, current directory, history, and my teammates' subsystems).
- errors.py and ui.py: shared error classes and colored output.

The built-in commands are help, echo, whoami, date, uptime, version, clear, history, and exit. I also added Windows-style aliases such as dir, cls, del, and type.

[Insert screenshots of the boot screen and a few commands here.]

3. How the Shell Works

The shell repeats one loop until the user types exit: read a line, split it into a command and arguments, look the command up in a dictionary called COMMANDS, run it, and handle any error. I used `shlex.split` so quoted text stays together as one argument.

Instead of a long chain of if/elif statements, I made a @command decorator that adds a function to the dictionary. This lets my teammates add commands in their own files without editing shell.py, which avoids conflicts. Every command uses the same signature, `(kernel, args)`.

The Kernel class is not a real kernel. It only holds shared data, but it plays a central role like a real kernel does. Because every command receives it, subsystems can work together without importing each other's files. For example, a process command can ask the memory manager for space through kernel.memory.

4. Error Handling

I made a base class called MiniOSError in errors.py, with smaller classes such as UsageError, PermissionDenied, OutOfMemory, and InvalidPID. My teammates raise these errors, and the shell shows them as a clean Error: <message> instead of a Python traceback. The shell also handles unknown commands, wrong arguments, unclosed quotes, and Ctrl+C, so it never crashes on bad input. I tested these cases along with empty input and capital letters.

5. OS Concepts Demonstrated

- **Command-line interface and shell:** the user gives instructions to the system by typing commands.
- **Simulated system calls:** each command is a request that the shell passes to the right part of MiniOS.
- **Error handling:** invalid requests are reported clearly without stopping the system.
- **System information:** uptime, date, whoami, and version show details about the running system.
- **Modular design:** separate parts communicate through one shared object.

6. Conclusion

Through this project I learned how a shell connects a user to the parts of an operating system. I also learned about circular imports, which is why our modules talk through the shared object instead of importing each other. MiniOS is limited because it has no pipes or output redirection, but it shows the basic ideas from our subject in a simple, working program.